

- Description

This module manages a smart grow light system for plant cultivation, ensuring that:

1. The system continuously monitors the current light intensity (in lumens).
2. If the light intensity falls below a predefined minimum threshold, the system increases the light brightness to enhance plant growth.
3. If the light intensity exceeds a predefined maximum threshold, the system decreases the light brightness to prevent energy waste and potential plant stress.
4. If the light intensity is within the optimal range, the system maintains an intermediate brightness level.

This system helps optimize indoor plant growth by maintaining ideal light conditions.

- Specification

module SmartGrowLightControl

var currentIntensity, minOptimalIntensity, maxOptimalIntensity, brightness: real

process updateIntensity(newIntensity: real) returns (currentIntensity: real, brightness: real)

functional scenario form:

currentIntensity < minOptimalIntensity && brightness = 100.0, output "ALERT: Light intensity (" + currentIntensity + " lumens) is too low. Brightness set to 100%.", Brightness increased to 100%.

||

currentIntensity > maxOptimalIntensity && brightness = 50.0, output "ALERT: Light intensity (" + currentIntensity + " lumens) is too high. Brightness set to 50%.", Brightness reduced to 50%.

||

currentIntensity <= maxOptimalIntensity && brightness = 75.0, output "Optimal light intensity (" + currentIntensity + " lumens). Brightness maintained at 75%.", Brightness set to 75%.

end process

end module

- Java Code

```
package code;
public class c102_SmartGrowLight {
    private double minOptimalIntensity;
    private double maxOptimalIntensity;
    private double currentIntensity;
    private double brightness;
    public c102_SmartGrowLight(double minOptimalIntensity, double maxOptimalIntensity, double initialIntensity, double initialBrightness) {
        this.minOptimalIntensity = minOptimalIntensity;
        this.maxOptimalIntensity = maxOptimalIntensity;
        this.currentIntensity = initialIntensity;
```

```

        this.brightness = initialBrightness;
    }
    public void updateIntensity(double newIntensity) {
        currentIntensity = newIntensity;
        adjustBrightness();
        System.out.printf("Current Light Intensity: %.2f lumens, Brightness: %.2f%%\n", currentIntensity, brightness);
        if (currentIntensity < minOptimalIntensity) {
            System.out.println("ALERT: Light intensity too low. Increasing brightness.");
        } else if (currentIntensity > maxOptimalIntensity) {
            System.out.println("ALERT: Light intensity too high. Decreasing brightness.");
        } else {
            System.out.println("Light intensity is optimal. Maintaining current brightness.");
        }
    }
    private void adjustBrightness() {
        if (currentIntensity < minOptimalIntensity) {
            brightness = 100.0;
        } else if (currentIntensity > maxOptimalIntensity) {
            brightness = 50.0;
        } else {
            brightness = 75.0;
        }
    }
    public double getCurrentIntensity() {
        return currentIntensity;
    }
    public double getBrightness() {
        return brightness;
    }
    public static void main(String[] args) {
        c102_SmartGrowLight growLight = new c102_SmartGrowLight(400.0, 800.0, 600.0, 75.0);

        // testcase-VT:
        //     growLight.updateIntensity(350.0);
        //     growLight.updateIntensity(750.0);
        //     growLight.updateIntensity(850.0);

        // testcase-FT:
        //     growLight.updateIntensity(9.063142030601998E290);
    }
}

```

```
//      growLight.updateIntensity(2.3292275018647134E293);
//      growLight.updateIntensity(6.7769769832148005E302);

//      // testcase-Z3:
//      growLight.updateIntensity(350.0);
//      growLight.updateIntensity(750.0);
//      growLight.updateIntensity(850.0);

//      // testcase-UVT:
//      growLight.updateIntensity(350.0);
//      growLight.updateIntensity(750.0);
//      growLight.updateIntensity(850.0);
    }
}
```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c102_SmartGrowLight.java	100.0 %	8	0	8
▼ c102_SmartGrowLight	100.0 %	8	0	8
● updateIntensity(double)	100.0 %	4	0	4
■ adjustBrightness()	100.0 %	4	0	4
● c102_SmartGrowLight(double, d		0	0	0
● main(String[])		0	0	0

3. Vibration testing instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	104	21,760	21,864
▼ src	0.5 %	104	21,760	21,864
▼ code	0.5 %	104	21,760	21,864
▼ c102_SmartGrowLight.java	94.5 %	104	6	110
▼ c102_SmartGrowLight	94.5 %	104	6	110
● updateIntensity(double)	100.0 %	47	0	47
■ adjustBrightness()	100.0 %	24	0	24
● c102_SmartGrowLight(double, d	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c102_SmartGrowLight.java	50.0 %	4	4	8
▼ c102_SmartGrowLight	50.0 %	4	4	8
● updateIntensity(double)	50.0 %	2	2	4
■ adjustBrightness()	50.0 %	2	2	4
● c102_SmartGrowLight(double, d		0	0	0
● main(String[])		0	0	0

5. Fuzz testing instruction coverage: 81.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	90	21,774	21,864
▼ src	0.4 %	90	21,774	21,864
▼ code	0.4 %	90	21,774	21,864
▼ c102_SmartGrowLight.java	81.8 %	90	20	110
▼ c102_SmartGrowLight	81.8 %	90	20	110
● updateIntensity(double)	85.1 %	40	7	47
■ adjustBrightness()	70.8 %	17	7	24
● c102_SmartGrowLight(double, d	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c102_SmartGrowLight.java	100.0 %	8	0	8
▼ c102_SmartGrowLight	100.0 %	8	0	8
● updateIntensity(double)	100.0 %	4	0	4
■ adjustBrightness()	100.0 %	4	0	4
● c102_SmartGrowLight(double, d		0	0	0
● main(String[])		0	0	0

7. Z3 instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	104	21,760	21,864
▼ src	0.5 %	104	21,760	21,864
▼ code	0.5 %	104	21,760	21,864
▼ c102_SmartGrowLight.java	94.5 %	104	6	110
▼ c102_SmartGrowLight	94.5 %	104	6	110
● updateIntensity(double)	100.0 %	47	0	47
■ adjustBrightness()	100.0 %	24	0	24
● c102_SmartGrowLight(double, d	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c102_SmartGrowLight.java	100.0 %	8	0	8
▼ c102_SmartGrowLight	100.0 %	8	0	8
● updateIntensity(double)	100.0 %	4	0	4
■ adjustBrightness()	100.0 %	4	0	4
● c102_SmartGrowLight(double, d		0	0	0
● main(String[])		0	0	0

9. Unimproved V-method instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	104	21,760	21,864
▼ src	0.5 %	104	21,760	21,864
▼ code	0.5 %	104	21,760	21,864
▼ c102_SmartGrowLight.java	94.5 %	104	6	110
▼ c102_SmartGrowLight	94.5 %	104	6	110
● updateIntensity(double)	100.0 %	47	0	47
■ adjustBrightness()	100.0 %	24	0	24
● c102_SmartGrowLight(double, d	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18